Class Notes RAG Assistant

Architecture & Design Document — Technical Evaluation Submission

A full-stack, retrieval-augmented study assistant that lets students upload their own notes and get grounded, cited answers — with web-search fallback and auto-generated quizzes.

Prepared by: Aakilathi
Track: AI (RAG) + Full Stack

1. Problem Statement

Students accumulate PDFs of class notes, syllabi, and question banks but have no fast way to query them. Generic AI chatbots answer from general knowledge, not the student's actual material, which risks hallucinated or irrelevant answers during exam prep.

Goal: build a system where a student uploads their own notes and gets answers grounded in that specific material, with a transparent indication of whether the answer came from their notes or a web fallback, and can self-test via auto-generated quizzes.

Scope for this submission

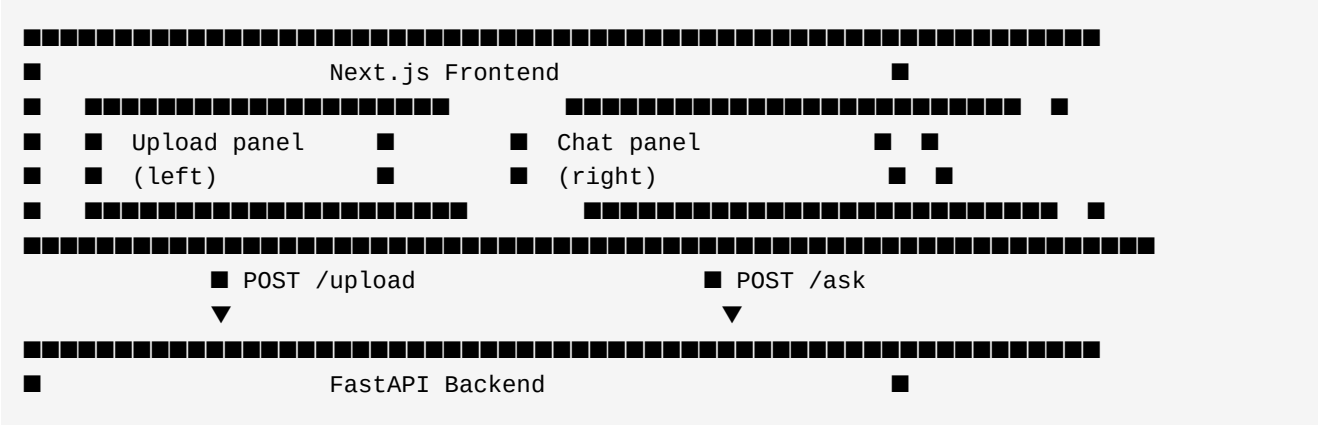
- Upload a PDF of notes → chunk → embed → store as vectors
- Ask a question → retrieve relevant chunks → generate a grounded, cited answer
- If the notes don't cover the topic, fall back to live web search, and say so explicitly
- Generate a short multiple-choice quiz from either source
- Single-page, minimal, typography-first UI — upload left, chat right

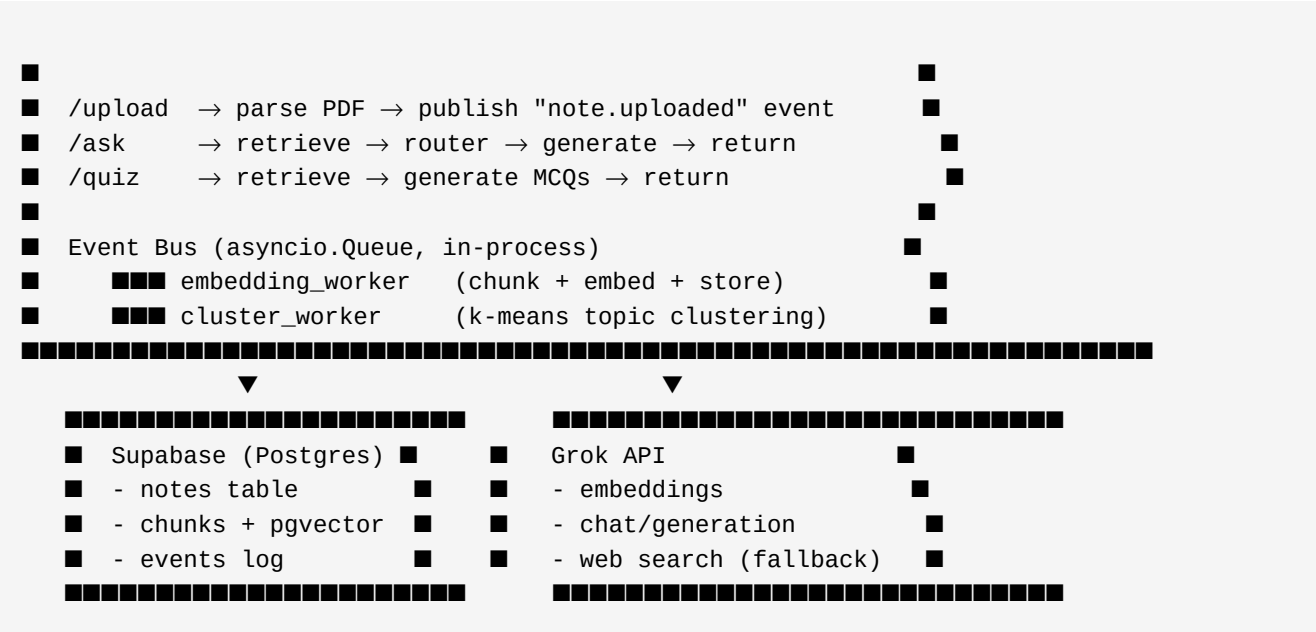
Explicitly out of scope

- Multi-turn conversational memory / long chat history
- User accounts, auth, multi-tenant isolation
- Production deployment, CI/CD, monitoring
- Supervised ML model training (uses existing embeddings for unsupervised topic clustering only, see §6)

2. System Architecture

The system is a single FastAPI backend serving a Next.js frontend, backed by Supabase (Postgres + pgvector) for both relational data and vector storage. An LLM API (Grok, with web-search capability) handles embeddings, generation, and quiz creation. An in-process event bus (asyncio.Queue) decouples the upload pipeline into background workers.





3. Data Flow

3.1 Upload flow (write path)

- Student uploads a PDF via the frontend upload panel
- FastAPI /upload endpoint saves the file, extracts text (pypdf/pdfplumber)
- Endpoint publishes a note.uploaded event to the queue and returns immediately (202-style response) — student sees 'Processing...'
- embedding_worker (background) consumes the event: splits text into chunks, calls the embeddings API per chunk, writes rows to a chunks table with a pgvector column
- cluster_worker (background) re-runs k-means over the student's chunk embeddings and updates a lightweight topics summary (used to show 'your notes cover: recursion, graphs, DP...')
- Frontend polls /notes/status or receives a final state update → shows 'Ready'

3.2 Ask flow (read path)

- Student types a question in the chat panel
- FastAPI /ask embeds the query, runs a pgvector similarity search (cosine distance) against that student's chunks
- Router step: if top-match similarity is above a threshold, treat notes as sufficient
- Sufficient → generate an answer grounded only in retrieved chunks, tag response source: 'notes', include page/unit citation
- Insufficient → call Grok's web search, generate an answer from web results, tag response source: 'web', and say so explicitly to the student

3.3 Quiz flow

- Student clicks 'Generate Quiz' (appears after at least one successful upload)
- Backend retrieves a spread of chunks across the student's detected topic clusters (not just top-3 for one query)
- LLM generates N multiple-choice questions with 4 options, correct answer, and a one-line explanation, grounded in the retrieved content
- Frontend renders the quiz inline in the right panel as a distinct message type (no page navigation)

4. Component Breakdown

Layer	Technology	Responsibility
Frontend	Next.js (React)	Single-page split layout: upload (left), chat (right). Minimal typography-first UI, no navigat
Backend API	FastAPI	/upload, /ask, /quiz endpoints. Publishes events, orchestrates retrieval + generation, neve
Event bus	asyncio.Queue (in-process)	Decouples upload from indexing. Chosen over Redis/Celery for zero extra infra on a sam
Database	Supabase (Postgres)	Stores notes metadata, chunk text, events log.
Vector store	Supabase pgvector extension	Stores chunk embeddings; cosine similarity search for retrieval. No separate vector DB n
LLM provider	Grok API	Embeddings, grounded answer generation, web search fallback, quiz generation — one
ML (unsupervised)	scikit-learn KMeans	Topic clustering over existing chunk embeddings — reuses RAG vectors, no separate tra

5. Design Decisions Worth Explaining

5.1 Typo / ambiguous query handling

Embeddings are inherently more typo-tolerant than keyword search since they match on semantic meaning, not exact strings. For domain-specific terms and acronyms embeddings handle less reliably, the query is first passed through the LLM as a rewrite step ('correct spelling/intent') before embedding — one cheap extra call that resolves the majority of failures.

5.2 Why asyncio.Queue instead of Redis/Celery

Redis or Celery is the more 'production-correct' choice for durability (jobs survive a restart) and horizontal scaling across multiple worker processes. For this scope and timeline, an in-process asyncio event bus demonstrates the identical architectural pattern — publish once, react independently, don't block the request — without added infrastructure risk. The swap to Redis is a drop-in change to the publish/subscribe layer only; no business logic changes.

5.3 Why the router step matters

Naively always searching the web defeats the purpose of RAG (students want answers grounded in their own material first). Naively never searching the web leaves gaps for anything not in their notes. A similarity-threshold router makes this decision explicit and explainable, and its outcome is always shown to the student (source-of-answer tag) rather than hidden.

5.4 Where ML fits without a separate track

Rather than bolting on an unrelated recommendation or anomaly-detection system, topic clustering (k-means) runs directly on the embeddings already produced for RAG — zero new data pipeline. This

is real unsupervised ML on real data, demoable as: 'your notes cover roughly N topics: ...' — and it reuses infrastructure rather than duplicating it.

6. Known Limitations & Future Work

- No persistence across server restarts for the event queue itself (would move to Redis Streams / Kafka in production)
- No user auth — chunks are not isolated per user in this build; production would need row-level security in Supabase per student
- Hybrid search (pgvector + pg_trgm fuzzy match) would catch typo cases the LLM rewrite step misses
- Quiz difficulty is currently LLM-labeled, not model-scored; a real difficulty classifier is a natural extension
- Payment/checkout, multi-turn memory, and spaced-repetition scheduling were deliberately scoped out to preserve depth over breadth

7. Summary

This system demonstrates RAG (retrieval grounded in user-owned data), an agentic routing decision (notes vs. web), generative AI beyond Q&A (quiz creation), a decoupled background-processing pattern (event bus + workers), and applied unsupervised ML (topic clustering) reusing the same embeddings — all inside one coherent, explainable, full-stack build rather than several shallow, disconnected features.